

# MediKiosk

## Complete Engineering, AI, Security, ABDM and Deployment Guide

SIH26047 — Patient Case-Taking Software  
All India Institute of Ayurveda, Ministry of Ayush

A practical build manual for turning the SIH problem statement into a secure, deployable clinical intake platform.

**Version:** September 2026

**Audience:** Student engineering team building an SIH solution with a path to real-world deployment.

**Important:** This is an engineering guide, not legal or clinical advice. Real hospital deployment requires institutional approval, security/privacy review, appropriate vendor agreements, current ABDM/NHA requirements, and qualified clinical oversight.

# 1. Executive Summary

MediKiosk should be built as a secure clinical intake platform, not as a medical chatbot. The patient uses a kiosk-style web application to identify themselves, provide consent, answer a structured history through voice and touch, complete an AYUSH-specific assessment when relevant, scan prior records, review the collected information and submit it. The doctor receives a structured case before consultation.

The official PS requires a patient-facing platform that captures history through natural spoken conversation and guided touch, digitizes physical medical documents, creates a physician-ready structured summary, and integrates with HIS/ABDM before consultation. It also calls out multilingual voice, accessibility, AYUSH/Dashavidha, OCR, red-flag detection, privacy and consent. ■filecite■turn1file0■L12-L15■

**Core principle:** AI prepares the case. The physician verifies the information and makes the clinical decision.

The architecture below is deliberately more granular than the four PS modules. This makes the system easier to build, test, replace and deploy.

## 2. What the PS Actually Requires

The PS describes a clinical history-taking bottleneck in high-volume Indian OPDs, fragmentation of paper records, and a missing patient-facing first-mile layer for ABDM. Existing registration systems generally capture demographic and appointment information rather than complete clinical history; mobile apps may exclude elderly and low-literacy patients; manual history desks do not scale; and generic scanners do not extract or organize clinical content.

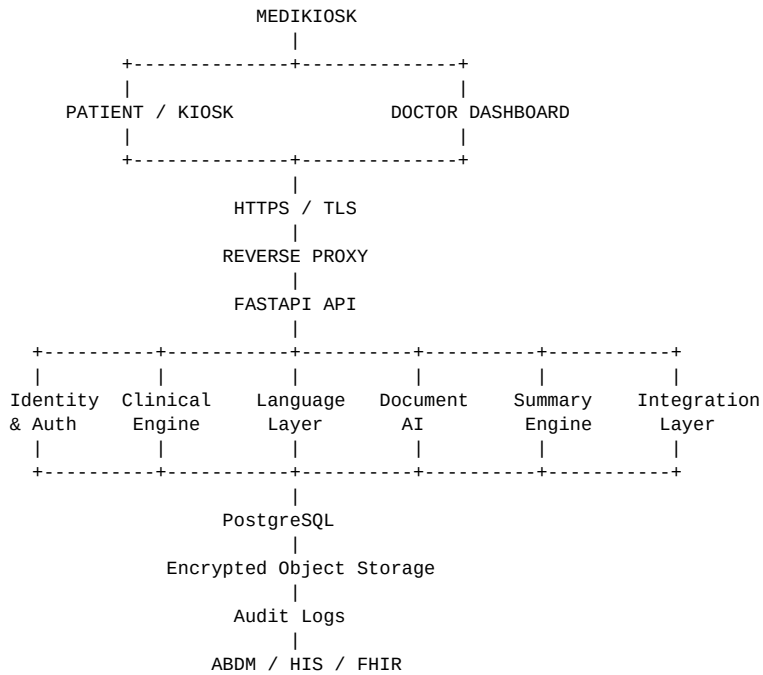
■filecite■turn1file0■L11-L13■

| PS requirement         | Our engineering interpretation                            |
|------------------------|-----------------------------------------------------------|
| Comprehensive history  | A controlled clinical state machine + question bank       |
| Multilingual voice     | ASR + translation/language handling + TTS                 |
| Voice + touch          | Every supported question has an accessible non-voice path |
| AYUSH history          | Dedicated structured Ayurveda assessment engine           |
| Documents              | Webcam capture + image preprocessing + OCR + extraction   |
| Physician-ready output | Typed clinical schema + summary + provenance              |
| Red flags              | Rules/clinical safety layer, not autonomous diagnosis     |
| Privacy/consent        | Cross-cutting security and consent architecture           |
| ABDM/HIS               | Adapter layer with mock, sandbox and production modes     |

The PS itself defines the patient journey as Identify → Converse → Scan → Summarize & Route → Consult.

■filecite■turn1file0■L14-L14■

### 3. Final Modular Architecture



Security and privacy sit across every layer: authentication, authorization, consent, encryption, data minimization, pseudonymization, auditability, secure sessions, secrets management, backups and monitoring.

## 4. Complete Module Map

| Module                   | What we build                                            | PS coverage                             |
|--------------------------|----------------------------------------------------------|-----------------------------------------|
| M1 Patient/Kiosk         | Elder-friendly UI, language, voice/touch, progress, help | Patient-facing software + accessibility |
| M2 Identity              | Patient UUID, ABHA mapping, visit IDs, registration      | Identify + ABHA                         |
| M3 Security              | Consent, RBAC, encryption, audit, session isolation      | Privacy, consent, security              |
| M4 Language              | Bhashini ASR/NMT/TTS adapter                             | Multilingual voice                      |
| M5 Clinical              | Question bank, state machine, HPI/PMH/ROS, red flags     | Adaptive history                        |
| M6 AYUSH                 | Dashavidha + Ahara/Vihara structured capture             | AYUSH requirement                       |
| M7 Document Capture      | Webcam scan, crop, perspective correction                | Physical records                        |
| M8 Document Intelligence | OCR, extraction, investigations, timeline                | Module B                                |
| M9 Summary               | Structured summary, provenance, uncertainties            | Module C                                |
| M10 Doctor               | Queue, case review, verification, consultation           | Physician workflow                      |
| M11 Integration          | FHIR-compatible model, ABDM/HIS adapters                 | Module D                                |
| M12 Operations           | Docker, CI/CD, monitoring, backups, deployment           | Deployability                           |

## 5. Patient Journey

### WELCOME

- > LANGUAGE
- > IDENTIFY
- > ABHA / DEMO AUTH
- > CONSENT
- > BASIC DETAILS
- > READY
- > CHIEF COMPLAINT
- > ADAPTIVE HISTORY
- > AYUSH (if applicable)
- > MEDICATION / ALLERGY
- > DOCUMENT SCAN
- > OCR / EXTRACTION
- > PATIENT REVIEW
- > SUBMIT
- > SECURE SESSION TERMINATION
- > DOCTOR QUEUE

Every state must be recoverable. If a network request fails, the application should not silently create another patient or another visit. Use a Visit ID, idempotency keys for critical submissions, and explicit status transitions.

The patient interface should feel like an ATM or railway-service kiosk: one task at a time, large controls, clear feedback, audio assistance, minimal typing and no conventional website navigation.

## 6. Doctor Journey

```
DOCTOR LOGIN
|
PATIENT QUEUE
|
OPEN CASE
|
+-- Patient Overview
+-- AI Clinical Summary
+-- Structured History
+-- AYUSH Assessment
+-- Medical Timeline
+-- Investigations
+-- AI / Red-Flag Flags
+-- Original Documents
|
VERIFY / EDIT
|
EXAMINATION
|
DIAGNOSIS / REASONING
|
TREATMENT / PRESCRIPTION
|
FINALIZE
```

A useful feature is source traceability. A doctor should be able to ask: “Where did this fact come from?” and see the original patient response or document page.

This is an important safety mechanism because a fluent AI summary can still be wrong.

## 7. Security & Privacy — Complete Model

| Control           | Implementation                                               |
|-------------------|--------------------------------------------------------------|
| Patient identity  | Internal random UUID; ABHA is an external identity reference |
| Authentication    | Separate patient, doctor and admin authentication            |
| Authorization     | RBAC + resource-level patient/visit checks                   |
| Consent           | Purpose, scope, version, timestamp, status, audit            |
| Encryption        | TLS in transit; encrypted DB/storage/backups                 |
| Data minimization | Send only necessary clinical context to AI                   |
| Document security | Private object storage; controlled access                    |
| Session isolation | Invalidate tokens and clear kiosk state                      |
| Audit             | Sensitive access/edit/finalize events                        |
| Secrets           | Server-side secret manager/environment                       |
| Monitoring        | Auth failures, anomalies, system health                      |
| Retention         | Documented institutional retention/deletion policy           |

ABDM's Health Data Management Policy describes “Security and Privacy by Design” and a federated architecture. NHA also describes consent-based exchange and decentralized storage of health records.

■cite■turn4search25■turn3search1■



## 8. Threat Model

| Threat                | Example                                  | Control                                                 |
|-----------------------|------------------------------------------|---------------------------------------------------------|
| Broken access control | Doctor changes patient UUID in URL       | Backend object authorization                            |
| Kiosk leakage         | Patient B sees Patient A                 | Session reset + no persistent client records            |
| Secret theft          | LLM key visible in browser               | Backend-only provider calls                             |
| Wrong patient         | Report attached to wrong visit           | Patient/visit ownership checks                          |
| OCR error             | 9.2 read as 92                           | Confidence + validation + source review                 |
| AI hallucination      | Diagnosis invented                       | Extractive schema + provenance + physician verification |
| Prompt injection      | Document tells AI to ignore instructions | Treat source text as untrusted data                     |
| Malicious upload      | Executable disguised as PDF              | Type/size validation + scanning                         |
| DB exposure           | Public PostgreSQL port                   | Private network + firewall                              |
| Log leakage           | Raw patient transcript in logs           | Structured safe logs                                    |

Rule: every frontend value, uploaded file, model output and external API response is untrusted until validated.

## 9. Database Design

```
patients
patient_identities
users
roles
consents
visits
conversation_sessions
clinical_histories
ayush_assessments
documents
document_extractions
medications
investigations
timeline_events
clinical_summaries
physician_reviews
audit_logs
```

### Core relationships:

```
Patient
|
+-- Identity mappings
+-- Consents
+-- Visits
    |
    +-- Conversation
    +-- Clinical History
    +-- AYUSH Assessment
    +-- Documents -> Extractions
    +-- Investigations
    +-- Timeline
    +-- Summary -> Physician Review
    +-- Consultation
```

Every clinical object should carry `patient_id` and, where it belongs to a specific encounter, `visit_id`. This prevents cross-visit and cross-patient mixing.

## 10. Database Fields That Matter

```
patients:
  id UUID PRIMARY KEY
  status
  created_at

patient_identities:
  id
  patient_id
  type # ABHA / hospital
  external_ref
  verified_at

visits:
  id UUID PRIMARY KEY
  patient_id
  department
  status
  created_at

consents:
  id
  patient_id
  visit_id
  purpose
  scope_json
  version
  status
  granted_at
  revoked_at

documents:
  id
  patient_id
  visit_id
  storage_key
  type
  sha256
  status
  created_at

clinical_summaries:
  id
  visit_id
  summary_json
  model_version
  status

physician_reviews:
  id
  summary_id
  physician_id
  changes_json
  verified_at

audit_logs:
  id
  actor_id
  actor_role
  patient_id
  visit_id
  action
  resource_type
  resource_id
  timestamp
  metadata_json
```

**Do not use ABHA as the internal primary key. Do not store Aadhaar merely because it was used in an authentication pathway.**

## 11. API Architecture

| Endpoint                          | Purpose                                  |
|-----------------------------------|------------------------------------------|
| POST /auth/session                | Create authenticated application session |
| POST /patients                    | Create patient                           |
| POST /visits                      | Create visit                             |
| POST /consents                    | Record consent                           |
| POST /conversation/session        | Start case-taking                        |
| POST /conversation/answer         | Submit answer                            |
| GET /conversation/state           | Get current state                        |
| POST /documents/upload            | Secure document upload                   |
| POST /documents/{id}/process      | Queue processing                         |
| GET /documents/{id}/original      | Authorized source access                 |
| GET /visits/{id}/summary          | Doctor-authorized summary                |
| PATCH /summaries/{id}             | Edit draft                               |
| POST /summaries/{id}/verify       | Verify summary                           |
| POST /consultations/{id}/finalize | Finalize consultation                    |
| GET /health                       | Service health                           |

Sensitive APIs should execute: authenticate → authorize → validate → business rule check → database operation → audit event.

## 12. Frontend Architecture

Recommended: React + TypeScript + Tailwind CSS. React Router handles route/state boundaries. TanStack Query or an equivalent request layer handles server state. React Hook Form + Zod can be used for structured forms. Browser MediaDevices handles camera access; MediaRecorder/Web Audio handles microphone capture.

```
src/  
  app/  
  components/  
  features/  
    patient/  
    conversation/  
    ayush/  
    documents/  
    doctor/  
    auth/  
    consent/  
  pages/  
    patient/  
    doctor/  
  services/  
    api.ts  
    auth.ts  
    language.ts  
    documents.ts  
  types/  
    patient.ts  
    visit.ts  
    clinical.ts  
    ayush.ts
```

**Do not put medical records, LLM keys, Bhashini keys or ABDM client secrets in browser bundles or localStorage.**

## 13. Backend Architecture

Recommended: Python + FastAPI + PostgreSQL + SQLAlchemy + Pydantic. Add Redis and Celery/RQ when OCR/AI processing needs background jobs.

```
backend/  
  app/  
    main.py  
    config.py  
    api/routes/  
    core/  
      security.py  
      permissions.py  
      audit.py  
      logging.py  
    models/  
    schemas/  
    repositories/  
    services/  
      clinical_engine.py  
      ayush_engine.py  
      bhashini_service.py  
      llm_service.py  
      document_service.py  
      summary_service.py  
      abdm_service.py  
    workers/
```

Keep route handlers thin. Put business rules in services and database operations in repositories. This makes the system testable and makes external integrations replaceable.

## 14. AI Architecture

```
Patient speech/touch
|
Language service
|
Clinical state + rules
|
LLM / NLP interpretation
|
Typed clinical facts
|
Validation + provenance
|
Next question / summary
```

Do not build a free-running medical chatbot. The clinical state machine decides what information is missing. The LLM interprets natural language and produces structured information.

Use structured outputs whenever possible. Google's Gemini documentation recommends structured outputs plus application-side validation; OpenAI provides Structured Outputs; Anthropic supports structured outputs/tool contracts.

■cite■turn5search9■turn5search5■turn5search6■

Never let model prose become the authoritative medical record.

## 15. Clinical Conversation Engine

Build a question bank. Each question needs an ID, section, purpose, answer type, required flag, language variants, validation rules and branching rules.

```
Question {
  id
  section
  purpose
  type
  required
  options
  language_variants
  conditions
  next_rules
}

ConversationState {
  visit_id
  current_section
  current_question
  answers
  missing_required_fields
  red_flags
}
```

### Example:

```
Chief complaint = chest pain
-> onset
-> character
-> location
-> radiation
-> severity
-> breathlessness
-> sweating
-> fainting
-> relevant history
```

The LLM can interpret “kal raat se dard hai” as onset information. It should not be allowed to decide that the entire HPI is complete without the clinical engine checking required fields.



## 16. Red-Flag Safety Engine

Use deterministic rules for obvious high-priority patterns. A red flag should trigger attention, not an autonomous diagnosis.

Example:

```
IF chest_pain == present
AND breathlessness == present
AND sweating == present
THEN
    create_priority_alert()
    notify_triage_staff()
    record_reason()
```

**The system should show the clinician what triggered the flag. Example: “Priority attention: chest pain + breathlessness reported.” Avoid wording such as “AI diagnosed heart attack.”**

For a real deployment, red-flag rules must be reviewed and approved by qualified clinicians and institutional triage policy.

## 17. AYUSH / Ayurveda Engine

The PS explicitly identifies the additional complexity of Ayurvedic history and asks for Dashavidha Pariksha and Ahara-Vihara assessment. ■filecite■turn1file0■L11-L13■

```
AYUSH MODE
|
+-- Dashavidha Pariksha
|   +-- Prakriti
|   +-- Vikriti
|   +-- Sara
|   +-- Samhanana
|   +-- Pramana
|   +-- Satmya
|   +-- Satva
|   +-- Ahara Shakti
|   +-- Vaya
|
+-- Ahara
+-- Vihara
+-- Agni
+-- Koshtha
+-- Nidana-related information
+-- Samprapti-related information
```

**Use the PS as the requirements source. For clinical definitions, wording and decision rules, work with a qualified Ayurveda/AYUSH subject-matter expert. The software captures and structures information; it does not autonomously prescribe.**

## 18. Document Scanning

The SIH prototype can use a laptop webcam as the document scanner. The live preview can be mirrored for user comfort, but the captured image sent to OCR must be correctly oriented and must not be mirrored.

Webcam

- > live preview
- > document boundary
- > capture
- > crop
- > perspective correction
- > rotation/orientation
- > image enhancement
- > OCR
- > extraction
- > storage

Store the original document separately from OCR text and extracted entities. The doctor must be able to open the original source when an OCR value looks suspicious.

## 19. Document Intelligence

OCR is only the first step. The PS asks for intelligent extraction of diagnoses, medications, dosages, investigation values and procedure history, plus chronological organization and abnormal-value highlighting.

■filecite■turn1file0■L14-L14■

```
Document
|
OCR text + bounding boxes
|
Medical entity extraction
|
+-- diagnosis
+-- medication
+-- dosage
+-- frequency
+-- investigation
+-- value
+-- unit
+-- date
+-- procedure
|
Validation
|
Timeline
```

### Example structured observation:

```
{
  "test_name": "Hemoglobin",
  "value": 9.2,
  "unit": "g/dL",
  "observed_at": "2026-08-12",
  "source_document_id": "doc_123",
  "confidence": 0.94
}
```

## 20. Clinical Summary Engine

Inputs:

- conversation\_facts
- ayush\_assessment
- document\_facts
- timeline\_events

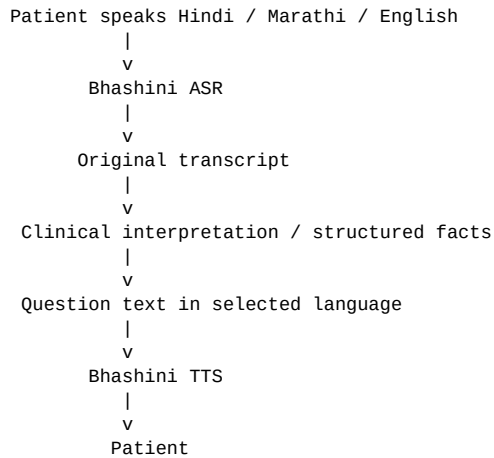
Output:

- chief\_complaint
- hpi
- past\_medical\_history
- past\_surgical\_history
- medications
- allergies
- family\_history
- personal\_history
- ros
- prior\_investigations
- ayush\_assessment
- timeline
- flags
- uncertainties
- sources

Every important fact should have provenance where possible: patient response, document ID/page, extraction model, timestamp, confidence and verification status.

The doctor should see three concepts separately: **source fact**, **AI interpretation**, and **physician-verified fact**.

## 21. Multilingual Architecture with Bhashini



The PS specifically mentions Indian-language ASR and Bhashini/AI4Bharat-type models.

■filecite■turn1file0■L12-L13■

The clinical schema should remain language-independent. Hindi, Marathi and English responses should populate the same structured fields.

## 22. Bhashini API — How to Get Credentials

The current Bhashini API documentation describes onboarding through the ULCA/Bhashini portal. After email verification and login, the integrator opens My Profile and generates application-specific API keys. The documentation currently states that an integrator can create up to five keys and obtains a User ID from the profile.

■cite■turn2search0■

[Official Bhashini onboarding documentation](#)

### Steps:

- Open the registration link from the official documentation.
- Create the integrator account and verify the email.
- Log in and open My Profile.
- Generate an API key for the MediKiosk application.
- Copy the User ID and ULCA API key into backend secrets.
- Never put the key in React/browser code.
- Search/select a pipeline for ASR, NMT and/or TTS.
- Use the current pipeline configuration and compute-call flow.

Bhashini's public API documentation also notes that the documented APIs are intended for PoC and asks production/charged use cases to contact the Bhashini team for the appropriate production version/pricing.

■cite■turn9search1■

## 23. Bhashini API — Technical Flow

The documented flow is Pipeline Search (optional) → Pipeline Config (mandatory) → Pipeline Compute (mandatory).

■cite■turn9search1■

### The documented Pipeline Config endpoint is:

<https://meity-auth.ulcacontrib.org/ulca/apis/v0/model/getModelsPipeline>

The config call uses headers such as **userID** and **ulcaApiKey**. Its response contains the inference callback URL and an inference API-key name/value used for the compute call. ■cite■turn2search1■turn9search3■

```
# backend environment
BHASHINI_USER_ID=...
BHASHINI_ULCA_API_KEY=...

class BhashiniClient:
    def get_pipeline_config(self, pipeline_id, tasks):
        # POST current config endpoint
        # send userID + ulcaApiKey
        # return current provider response
        pass

    def compute(self, callback_url, auth_name, auth_value, payload):
        # POST to callback_url using returned auth header
        pass
```

**Do not copy authentication values from examples in public documentation. The example response contains sample values; your account receives its own values. ■cite■turn9search3■**



## 24. Bhashini Adapter Design

```
LanguageProvider
|
+-- MockLanguageProvider
|
+-- BhashiniProvider
|
+-- FutureLocalProvider

Methods:
transcribe(audio, language)
translate(text, source, target)
synthesize(text, language)
```

This abstraction is important because external providers change. The clinical engine should not know whether speech came from Bhashini, a local model or another approved provider.

### Add a feature flag such as:

```
LANGUAGE_PROVIDER=mock
# or
LANGUAGE_PROVIDER=bhashini
```

## 25. LLM Choice — Recommended Setup

| Provider          | Good use                                           | Recommendation        |
|-------------------|----------------------------------------------------|-----------------------|
| Gemini            | Multimodal input, structured extraction, reasoning | Primary SIH candidate |
| OpenAI            | Structured extraction, tool/function workflows     | Strong alternative    |
| Claude            | Structured outputs, tool contracts, reasoning      | Alternative/reviewer  |
| Self-hosted model | On-premise future deployment                       | Evaluate after MVP    |

As of September 2026, Google's current model list identifies Gemini 3.8 Flash as stable. Its model page lists text, image, video, audio and PDF inputs and structured outputs/function calling. [cite](#) [turn6search0](#) [turn6search3](#)

Recommendation: start with one model, behind an adapter. Do not add three providers merely for appearance. Benchmark the exact clinical extraction and language tasks first.

## 26. Gemini API — Setup

Google AI Studio is the quickest route to experiment with Gemini and obtain an API key. Google's current documentation provides a key-management page and AI Studio quickstart.

■cite■turn5search2■turn6search11■turn6search12■

[Google AI Studio](#)

[Gemini API key documentation](#)

```
GEMINI_API_KEY=...  
LLM_PROVIDER=gemini  
LLM_MODEL=gemini-3.8-flash
```

**Never use a browser-exposed variable such as VITE\_GEMINI\_API\_KEY. Provider calls must go through our backend.**

Google's current API-key guidance says new AI Studio keys are authorization keys and that unrestricted standard keys are being rejected as the API moves toward authorization keys. ■cite■turn5search2■

## 27. OpenAI and Claude — Optional Providers

OpenAI states that API/business inputs and outputs are not used for training by default for business users, and describes encryption and retention controls for qualifying organizations. ■cite■turn5search0■turn5search1■

### OpenAI business data privacy

```
OPENAI_API_KEY=...  
LLM_PROVIDER=openai  
LLM_MODEL=<approved current model>
```

Anthropic's current platform supports structured outputs and tool use. ■cite■turn5search3■turn5search6■

```
ANTHROPIC_API_KEY=...  
LLM_PROVIDER=anthropic  
LLM_MODEL=<approved current model>
```

**For actual patient data, provider selection must be based on institutional privacy/security requirements, contractual terms, retention, data residency/processing requirements and clinical risk review—not model benchmarks alone.**

## 28. ABDM — What It Is and What It Is Not

ABDM is an interoperable digital-health ecosystem. NHA describes it as enabling different healthcare systems to communicate through common standards and APIs. NHA also describes federated architecture, where records remain with the originating provider and are shared with intended stakeholders through consent.

■cite■turn0search1■turn3search1■

### Therefore:

- ABHA is an identity/health-account mechanism, not our internal database.
- ABDM is not a central database where we should dump our entire application database.
- ABHA entry does not mean automatic access to every medical record.
- Record sharing requires the applicable consent/interoperability flow.
- Our hospital database remains the operational source for the MediKiosk visit unless the deployment architecture says otherwise.

## 29. ABDM Sandbox — How to Get Started

NHA's current FAQ says digital health solution providers can integrate through the ABDM Sandbox and that production entry requires functional testing and security assessment. The FAQ also describes eligibility requirements and safeguards for sandbox participants. ■cite■turn0search1■turn7search4■

[ABDM official website](#)

[ABDM integration/partner information](#)

### Practical process:

1. Coordinate with an eligible institutional/registered entity.
2. Apply to ABDM Sandbox.
3. Receive sandbox credentials/client information.
4. Read the current API documentation for your assigned integration.
5. Build against sandbox.
6. Complete required functional testing.
7. Complete required security assessment.
8. Submit exit documentation.
9. Only after approval, use production integration.

**A student team should not assume that an individual student account automatically qualifies for production integration. If necessary, coordinate through the college/institute or an eligible registered health-tech entity.**

## 30. ABHA Authentication — Correct Engineering Approach

Do not invent an ABHA login flow. ABHA authentication is not simply “enter ABHA number and query a database.” Use the current ABDM sandbox authentication/registration APIs available to your integration client.

NHA's current material describes ABHA login/registration APIs and lists possible user entry points including mobile number/email registered with ABHA, ABHA number and ABHA address. ■cite■turn3search0■

Patient

- > choose ABHA
- > enter identifier required by current ABDM flow
- > complete authorized authentication/OTP step
- > ABDM returns current authentication result/token
- > backend validates result
- > map external identity -> Patient UUID
- > create Visit UUID

**Do not store OTPs. Do not implement your own Aadhaar authentication. Do not invent callback URLs, token endpoints or headers. The exact API sequence must come from the current ABDM sandbox documentation for your client/version.**

If credentials are unavailable, implement `MockABHAProvider` with the exact same internal interface and clearly label the UI as Demo/Sandbox.

## 31. ABDM Consent and Record Sharing

Keep two concepts separate: local MediKiosk intake consent and an ABDM record-sharing consent transaction. The former covers collection/processing in MediKiosk; the latter covers access/sharing through the ABDM ecosystem.

Local consent:  
  purpose = clinical intake  
  scope = history + uploaded documents

ABDM sharing:  
  purpose = share specified records with specified requester  
  scope = approved record types / duration / requester

NHA states that health records are shared with other doctors/facilities only after the patient's consent, and that ABDM facilitates secure exchange rather than centrally storing all records. ■cite■turn3search1■



## 32. FHIR / HIS Integration

Build a canonical internal clinical model first. Then map it to the hospital's actual FHIR/HIS contract. This avoids coupling the entire product to one hospital interface.

```
Internal Clinical Model
|
+--> FHIR Mapper
|
+--> Hospital HIS Mapper
|
+--> PDF/Export Mapper
```

Depending on the actual integration, useful FHIR resources may include Patient, Encounter, Observation, Condition, Medication-related resources, AllergyIntolerance, DiagnosticReport and DocumentReference. Use only the resources required by the approved integration.

NHA's FAQ lists standards such as SNOMED CT, LOINC, ICD-10 and DICOM as examples in digital-health interoperability. [■cite■turn0search1■](#)

## 33. Security Implementation — API and Session Controls

```
Sensitive request
|
Authenticate
|
Authorize role
|
Authorize patient/visit/resource
|
Validate input
|
Apply business rules
|
Perform operation
|
Write audit event
|
Return minimum necessary data
```

Use short-lived sessions/tokens where appropriate, secure cookies if that is the chosen session model, strict CORS, secure headers, rate limits, request-size limits and server-side validation.

For a shared kiosk, the session must have a clear maximum lifetime and explicit termination. After submission, the kiosk returns to a clean welcome state.

## 34. Secure File Uploads

### Upload

- > size check
- > MIME/type check
- > extension check
- > malware/security scan where appropriate
- > generate internal object name
- > private encrypted storage
- > queue processing

**Never use the patient-provided filename as a trusted server path. Never expose a permanent public URL for a medical document.**

Recommended document metadata: document\_id, patient\_id, visit\_id, type, storage\_key, hash, upload timestamp, processing status, OCR status, extraction status and source provenance.

## 35. AI Data Minimization

The AI provider should receive the minimum context required for the task.

```
BAD:
Entire patient record
+ name
+ phone
+ address
+ ABHA
+ every old document
+ unrelated visits
  |
  v
External LLM

BETTER:
Task-specific clinical context
+ only required source facts
+ pseudonymous internal reference
  |
  v
LLM
  |
  v
Validated structured output
```

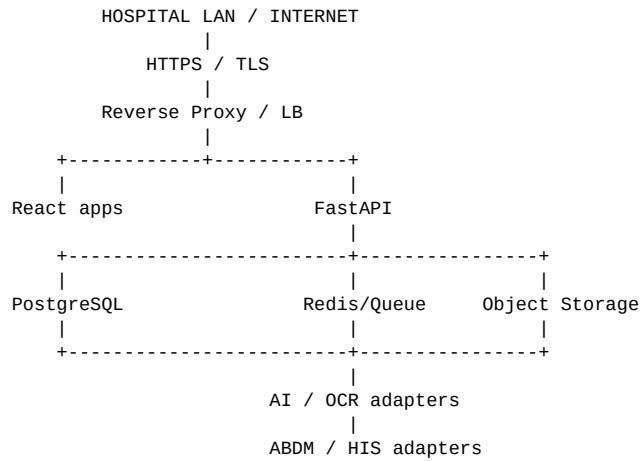
If the task is HPI extraction, the model does not need the patient's phone number. If the task is medication extraction, it may need only the relevant document and surrounding OCR text.

## 36. Data Protection and DPDP Practical Plan

| Area         | Engineering action                                        |
|--------------|-----------------------------------------------------------|
| Notice       | Simple privacy/consent explanation in local language      |
| Purpose      | Define why each category of data is collected             |
| Minimization | Do not collect unnecessary identifiers                    |
| Security     | Encryption, access controls, monitoring                   |
| Retention    | Institution-defined retention/deletion workflow           |
| Requests     | Process for applicable access/correction/erasure requests |
| Grievance    | Institutional contact/workflow                            |
| Processors   | Review cloud/AI/OCR vendors and contracts                 |
| Audit        | Keep evidence of sensitive processing/access              |

MeitY publishes the DPDP Act 2023 and DPDP Rules 2025. The 2025 Rules use phased commencement, so implementation status must be checked at the time of real deployment. ■cite■turn4search0■turn4search26■

## 37. Deployment Architecture



Multiple kiosks should be clients of the central application. The kiosk should not become the permanent storage system.

## 38. Docker Deployment

```
docker-compose.yml
|
+-- frontend
+-- backend
+-- postgres
+-- redis
+-- worker
+-- reverse-proxy
```

Use separate development, staging and production configurations. Production must not run in debug mode and must not expose PostgreSQL publicly.

### Example environment variables:

```
APP_ENV=production
DATABASE_URL=...
REDIS_URL=...
SESSION_SECRET=...
GEMINI_API_KEY=...
BHASHINI_USER_ID=...
BHASHINI_ULCA_API_KEY=...
ABDM_ENV=production
ABDM_CLIENT_ID=...
ABDM_CLIENT_SECRET=...
OBJECT_STORAGE_ENDPOINT=...
OBJECT_STORAGE_ACCESS_KEY=...
OBJECT_STORAGE_SECRET_KEY=...
```

**Never commit the real `.env` file.**

## 39. CI/CD and Release Process

```
feature branch
-> pull request
-> lint/type checks
-> unit tests
-> API tests
-> security scans
-> build
-> staging
-> E2E tests
-> human approval
-> production
```

**For healthcare software, autonomous agents should not have unrestricted production deployment permissions. Antigravity can prepare changes and verification artifacts, but production release should remain an approved human action.**



## 40. Testing Strategy

| Layer       | Test                                                    |
|-------------|---------------------------------------------------------|
| Unit        | Clinical branching, permissions, consent state, parsers |
| API         | Auth, patient isolation, document authorization         |
| Frontend    | Kiosk flow, accessibility, camera/mic failure           |
| AI          | Schema validity, hallucination, missing-data behavior   |
| OCR         | Printed/handwritten, rotation, low-quality samples      |
| Integration | Bhashini, LLM, ABDM mock/sandbox, HIS mock              |
| Security    | Broken access control, uploads, secrets, injection      |
| Load        | Multiple kiosks, queue spikes, OCR jobs                 |
| Recovery    | Network loss, worker failure, DB restore                |

Create a synthetic patient test pack. Never use real patient data for ordinary development or public demos.

## 41. Security Testing — Must Pass Before Pilot

- Patient A cannot access Patient B by changing a UUID.
- Doctor A cannot access an unauthorized patient.
- Logged-out users cannot access old documents.
- Patient B cannot see Patient A's kiosk state.
- Provider API keys are absent from browser source/network payloads.
- No public PostgreSQL port.
- Uploaded files cannot escape the intended storage path.
- Audit events exist for sensitive reads and writes.
- AI cannot directly finalize diagnosis/treatment.
- LLM/document prompt injection cannot override system rules.
- Expired sessions cannot access clinical data.
- Production secrets are absent from Git history.

ABDM production integration additionally requires the applicable functional testing and security assessment described by NHA. ■cite■turn7search4■

## 42. Stitch Workflow

Google describes Stitch as an AI-native design canvas for high-fidelity UI generation, iteration and prototyping, with export paths toward development tools. ■cite■turn8search0■turn8search1■turn8search2■

### [Google Stitch](#)

Use Stitch for design. Use AntigraVity for repository implementation, testing and integration.

Stitch:

1. Design system
2. Patient screens
3. Doctor screens
4. Prototype flow
5. Review
6. Export/hand-off

Antigravity:

1. Inspect repository
2. Plan
3. Implement
4. Test
5. Verify
6. Iterate

## 43. Stitch Master Prompt

Design a high-fidelity web application called MediKiosk for a government hospital clinical intake workflow in India.

The product prepares a structured medical case before consultation.

Two interfaces:

1. Patient/Kiosk
2. Doctor Dashboard

Patient requirements:

- kiosk/ATM-like simplicity
- elderly and low-literacy friendly
- Hindi, English and Marathi
- voice + touch
- audio instructions
- one question at a time
- large controls
- high contrast
- visible microphone states
- simple language
- help/staff assistance
- privacy/consent
- document scanning
- patient review before submission

Doctor requirements:

- queue
- patient overview
- AI clinical summary
- structured history
- AYUSH assessment
- medical timeline
- investigations
- original documents
- AI/red-flag flags
- source traceability
- edit/accept/reject/verify
- consultation workspace

Do not make this look like a consumer chatbot or wellness app.

Do not place a normal website navbar on the patient kiosk.

Create a reusable design system and consistent components.

## 44. Stitch Patient Screen Prompts

### Welcome:

Create the MediKiosk patient welcome screen.

Headline: "Let's prepare your medical case"

Subtext: "Answer a few questions before you meet the doctor."

Primary button: Start

Secondary: Need Help

Show language, audio/repeat, accessibility and privacy controls.

Use very large buttons and minimal text.

### Clinical question:

Create a patient clinical question screen.

Show exactly one question.

Show spoken-question/repeat control.

Provide a large "Speak" microphone button.

Provide large touch choices when appropriate.

Show Back only where allowed.

Show progress without overwhelming the user.

Include visual states:

idle, listening, processing, answer captured, retry.

### Document scan:

Create a document scanning screen for a laptop webcam.

Show a live document frame and simple instructions:

1. Place the document flat.
2. Keep the whole page visible.
3. Avoid shadows.
4. Hold still.

Buttons:

Capture Document

Retake

Use This Document

The captured document is not a mirrored selfie image.

## 45. Stitch AYUSH + Doctor Prompts

### AYUSH:

Create an AYUSH assessment screen for MediKiosk.

Capture structured Ayurveda information without looking like a paper form.  
Use one topic at a time, audio support and progress.

Topics:

Prakriti, Vikriti, Sara, Samhanana, Pramana, Satmya, Satva,  
Ahara Shakti, Vaya, Ahara, Vihara, Agni, Koshtha.

Do not generate diagnosis or treatment.

This screen collects information for physician review.

### Doctor dashboard:

Create a professional clinical dashboard for MediKiosk.

Navigation:

Dashboard

Patient Queue

Patients

AYUSH Assessments

Documents

Reports

Settings

Case view:

patient overview

AI summary

structured history

AYUSH assessment

medical timeline

investigations

AI/red-flag panel

original documents

source traceability

Actions:

Review Case

View Original Document

View Patient Response

Edit

Accept

Reject

Mark Verified

Add Clinical Note

Add Examination Finding

Finalize Consultation

## 46. Antigravity — How to Use It Professionally

Google Antigravity is an agentic development environment that can operate across editor, terminal and browser surfaces and produce plans, code changes, tests and artifacts. ■cite■turn1search0■turn1search2■

Use it as a supervised senior engineer, not as a one-shot code generator.

[Antigravity documentation](#)

### Recommended workflow:

- Create Git repository.
- Create the Antigravity Project around the repository.
- Add the SIH PS and this guide to the workspace.
- Ask for inspection and a plan before coding.
- Use isolated worktrees for risky changes where practical.
- Implement one milestone.
- Run tests.
- Review diffs/artifacts.
- Merge only after verification.

## 47. Antigravity Master Prompt

You are the lead software engineer for MediKiosk, a deployable clinical intake platform based on SIH26047 Patient Case-Taking Software.

DO NOT START CODING.

First inspect:

- repository
- SIH problem statement
- existing files
- current architecture

Then produce:

1. architecture assessment
2. folder structure
3. database entities
4. API boundaries
5. security model
6. AI/provider boundaries
7. first vertical slice
8. risks and assumptions
9. test plan

Technology baseline:

Frontend: React + TypeScript + Tailwind

Backend: Python + FastAPI

Database: PostgreSQL

ORM: SQLAlchemy

Validation: Pydantic

Queue: Redis + Celery/RQ

Storage: S3-compatible private object storage

Deployment: Docker

Requirements:

patient kiosk, doctor dashboard, multilingual voice/touch, adaptive history, AYUSH, red flags, webcam OCR, structured summary, physician verification, ABHA/ABDM boundary, consent, security, audit logging and deployability.

Non-negotiable:

- no secrets in frontend
- no direct DB access from frontend
- ABHA is not internal primary key
- every clinical object belongs to patient + visit
- no unnecessary Aadhaar storage
- no unnecessary PII to AI
- AI cannot finalize diagnosis/treatment
- validate AI output
- preserve provenance
- synthetic data in development
- external services have mock adapters
- do not claim live ABDM integration without proof

Do not modify code yet.



## 48. Antigravity — Phase Prompts

### Foundation:

Implement only the MediKiosk foundation.

Create:

- frontend shell
- FastAPI backend
- PostgreSQL
- SQLAlchemy
- Alembic
- Docker Compose
- .env.example
- health endpoint
- README
- basic CI

Acceptance:

docker compose starts  
frontend loads  
backend /health works  
database migration works  
no secrets hard-coded

### Identity + consent:

Implement patient identity and consent.

Create:

Patient UUID  
patient\_identities  
Visit UUID  
consents  
session model  
MockABHAProvider  
RBAC  
authorization middleware  
audit events

Write tests proving:

Patient A cannot access Patient B.  
A visit belongs to exactly one patient.  
Logged-out sessions cannot access clinical data.  
Consent is persisted and auditable.

Do not integrate live ABDM.

## 49. Antigravity — Patient and Doctor Vertical Slice

### Patient slice:

Implement:

Welcome -> Language -> Mock ABHA -> Consent -> Basic Details

-> Chief Complaint -> 5-10 deterministic questions -> Review -> Submit.

Use a deterministic question state machine first.

Use mock voice/text if needed.

Do not add an LLM yet.

Acceptance:

patient submission reaches FastAPI

visit is stored in PostgreSQL

doctor can retrieve it

patient session ends cleanly

tests pass

### Doctor slice:

Implement:

Doctor login

Patient queue

Patient overview

Structured history

Edit

Verify

Audit event

Do not implement diagnosis or treatment automation.

Add authorization tests.

## 50. Antigravity — LLM Adapter Prompt

Implement an LLM provider abstraction.

Methods:

```
extract_clinical_facts()
classify_answer()
generate_question_text()
generate_summary()
```

Adapters:

```
GeminiAdapter
OpenAIAAdapter
AnthropicAdapter
```

Configuration chooses one provider.

All extraction returns validated Pydantic models.

Use structured output.

Add timeout/retry/error handling.

Never log raw patient data.

Do not let provider-specific response shapes leak into the clinical engine.

Add unit tests for:

```
valid response
missing field
malformed response
provider timeout
provider refusal
semantic validation failure
```

## 51. Antigravity — Bhashini Prompt

Implement Bhashini as a backend language-service adapter.

Requirements:

- secrets backend-only
- environment configuration
- pipeline config
- cached configuration with safe expiry
- ASR
- translation
- TTS
- timeouts/retries
- provider error mapping
- English/Hindi/Marathi first
- MockLanguageProvider

Frontend calls our API:

POST /language/asr

POST /language/translate

POST /language/tts

Backend calls Bhashini.

Use the current official Bhashini documentation for exact payloads, headers and compute endpoint. Do not invent authentication values.

Feature flag:

LANGUAGE\_PROVIDER=mock|bhashini

## 52. Antigravity — OCR Prompt

Implement the medical document pipeline.

Frontend:

- camera permission
- live preview
- document boundary
- capture
- retake
- submit

Backend:

- secure upload
- type/size validation
- private storage
- SHA-256
- processing job
- OCR adapter
- medical extraction adapter
- investigation parser
- timeline generation

Store:

- original document
- OCR text
- structured extraction
- confidence
- provenance
- processing status

Doctor action:

- View Original Document

Do not expose permanent public document URLs.

## 53. Antigravity — Security Hardening Prompt

Perform a security hardening pass.

Audit:

- authentication
- authorization
- object-level access control
- session management
- CORS
- security headers
- rate limits
- file uploads
- SQL injection
- XSS
- CSRF where applicable
- secrets
- error leakage
- sensitive logging
- patient/visit isolation
- document access
- audit logging
- Docker configuration
- public DB exposure
- dependency vulnerabilities
- AI prompt injection

For every finding:

1. explain risk
2. identify file
3. implement fix
4. add regression test
5. report remaining limitations

Do not claim "fully secure".

Do not make unrelated refactors.

## 54. Antigravity — ABDM Prompt

Implement an ABDM integration boundary.

Create:

ABDMPProvider interface  
MockABDMPProvider  
SandboxABDMPProvider skeleton  
ProductionABDMPProvider skeleton  
ABDM\_ENV=mock|sandbox|production  
external identity mapping  
consent transaction model  
record request model  
integration audit events  
FHIR-compatible mapping layer

Important:

Use the current official ABDM sandbox documentation for exact authentication, endpoints, callbacks, headers and transaction sequence.

Do not invent endpoints.

Do not store OTPs.

Do not embed client secrets.

Do not automatically fetch records without the applicable consent flow.

Create a mock test:

Patient -> consent -> record request -> mock response -> doctor view.

## 55. Antigravity — Full E2E Test Prompt

Create a full synthetic-data end-to-end test.

Scenario:

1. Synthetic patient chooses Hindi.
2. Mock ABHA succeeds.
3. Consent is granted.
4. Patient answers history questions.
5. Red-flag scenario is detected.
6. Synthetic lab report is uploaded.
7. OCR returns expected values.
8. Summary is generated.
9. Doctor opens queue.
10. Doctor verifies/edits.
11. Doctor finalizes consultation.
12. Audit trail is generated.
13. Patient session terminates.

Negative tests:

- wrong patient
- expired session
- missing consent
- invalid document
- OCR failure
- LLM timeout
- Bhashini timeout
- duplicate submission
- network interruption

No real patient data.



## 56. Antigravity — Deployment Prompt

Prepare MediKiosk for deployment.

Create:

production Dockerfiles  
docker-compose local/staging  
reverse proxy config  
health checks  
graceful shutdown  
database migrations  
worker startup  
secure env configuration  
private document storage  
backup instructions  
safe logging  
CI pipeline

Create:

DEPLOYMENT.md  
SECURITY.md  
OPERATIONS.md  
DISASTER\_RECOVERY.md

Do not auto-deploy to production.

Report:

- local start
- staging start
- required secrets
- infrastructure
- known production blockers

## 57. Environment and Secret Checklist

| Secret/group | Examples                                | Rule                   |
|--------------|-----------------------------------------|------------------------|
| Database     | DATABASE_URL                            | Backend only           |
| Sessions     | SESSION_SECRET, JWT_SECRET              | Backend/secret manager |
| Gemini       | GEMINI_API_KEY                          | Backend only           |
| OpenAI       | OPENAI_API_KEY                          | Backend only           |
| Claude       | ANTHROPIC_API_KEY                       | Backend only           |
| Bhashini     | BHASHINI_USER_ID, BHASHINI_ULCA_API_KEY | Backend only           |
| ABDM         | ABDM_CLIENT_ID, ABDM_CLIENT_SECRET      | Backend only           |
| Storage      | OBJECT_STORAGE_ACCESS_KEY/SECRET        | Backend/worker only    |

**Use `.env.example` with blank placeholders. Keep actual secrets in environment/secret-management systems. Rotate keys if they are exposed.**

## 58. Production Operations

A deployable system needs more than application code.

- Health checks for API, database, queue and storage.
- Error monitoring.
- Application metrics and latency.
- Audit log monitoring.
- Backup success monitoring.
- Dependency/security update process.
- Incident response procedure.
- Access review for doctor/admin accounts.
- Key rotation procedure.
- Documented restore procedure.

**Avoid putting raw patient speech or complete medical summaries into ordinary application logs.**

## 59. Backup and Disaster Recovery

Primary PostgreSQL  
|  
encrypted automated backup  
|  
separate backup location  
|  
periodic restore test

### Define:

- **RPO:** maximum acceptable data loss measured in time.
- **RTO:** maximum acceptable restoration time.

Do not promise an RPO/RTO until the infrastructure actually supports it.

## 60. AI Safety Evaluation

| Metric                | What to measure                       |
|-----------------------|---------------------------------------|
| History completeness  | Required fields captured correctly    |
| Extraction precision  | How often extracted facts are correct |
| Extraction recall     | How often important facts are missed  |
| Unsupported-fact rate | Facts not present in source           |
| Red-flag sensitivity  | Important flags detected              |
| False positives       | Unnecessary alerts                    |
| Language accuracy     | ASR/translation quality               |
| Latency               | Question-to-response time             |
| Fallback success      | Touch path when AI fails              |

Every model or prompt change should be evaluated against the same synthetic benchmark. Do not judge the system only by whether one demo conversation sounds good.

## 61. Recommended SIH Demo

1. Patient opens MediKiosk.
2. Selects Hindi.
3. Uses Demo ABHA.
4. Gives consent.
5. Says: "Mujhe teen din se seene mein dard hai."
6. System asks adaptive follow-ups.
7. Red-flag attention is generated.
8. Patient scans a synthetic prescription/lab report.
9. OCR extracts medication/investigation/date.
10. Summary is generated.
11. Doctor opens the queue.
12. Doctor reviews summary and source document.
13. Doctor edits/marks verified.
14. Doctor completes consultation.
15. Show audit trail.
16. Show secure kiosk reset.

Final line:

"AI prepares the case. The physician makes the clinical decision."

Keep the demo synthetic and controlled. The purpose is to demonstrate architecture and workflow, not to simulate an uncontrolled medical deployment.

## 62. Demo Failure Plan

| Failure    | Fallback                             |
|------------|--------------------------------------|
| Internet   | Preloaded synthetic demo mode        |
| Bhashini   | Mock language provider / touch input |
| LLM        | Deterministic synthetic response     |
| OCR        | Prepared synthetic document/mock OCR |
| ABDM       | Mock provider labelled Demo/Sandbox  |
| Webcam     | Prepared synthetic image upload      |
| Microphone | Touch answers                        |

A deployable architecture should degrade gracefully rather than stop the clinical intake workflow when one external service is unavailable.

## 63. Exact Build Order

| Phase | Build                       | Exit condition                    |
|-------|-----------------------------|-----------------------------------|
| 0     | Accounts, repository, tools | Stitch/Antigravity/Git ready      |
| 1     | Architecture + DB           | Schemas approved                  |
| 2     | Backend                     | API + DB + migrations             |
| 3     | Patient UI                  | Mock end-to-end kiosk             |
| 4     | Doctor UI                   | Doctor sees submitted case        |
| 5     | Clinical engine             | Adaptive deterministic flow       |
| 6     | Voice                       | Bhashini adapter works            |
| 7     | AYUSH                       | Structured AYUSH flow             |
| 8     | Documents                   | Webcam + OCR + extraction         |
| 9     | Summary                     | Combined provenance-aware summary |
| 10    | Security                    | RBAC/session/audit tests          |
| 11    | ABDM                        | Mock then sandbox                 |
| 12    | Deployment                  | Docker/staging works              |
| 13    | Hardening                   | E2E/security/AI evaluation        |

Do not start with live ABDM. First prove the internal vertical slice.



## 64. The First Milestone — Start Here

```
Patient
-> Welcome
-> Language
-> Mock ABHA
-> Consent
-> Basic Details
-> Chief Complaint
-> 5-10 deterministic questions
-> Review
-> Submit
    |
    v
  FastAPI
    |
    v
  PostgreSQL
    |
    v
Doctor
-> Login
-> Queue
-> Open Case
-> Read Structured History
-> Verify
```

This milestone proves the fundamental product loop. Once it works, the external services become adapters rather than foundations.

## 65. Common Engineering Mistakes to Avoid

- Building the entire UI before defining the data model.
- Calling Bhashini directly from React.
- Putting LLM keys in ``.env`` variables exposed to the frontend.
- Using ABHA as the database primary key.
- Saving all medical records in browser `localStorage`.
- Letting the LLM decide the complete clinical interview.
- Letting AI-generated text become the final medical record automatically.
- Treating OCR output as truth without source verification.
- Hard-coding one external provider everywhere.
- Hard-coding old ABDM endpoints from tutorials.
- Using real patient data in development.
- Claiming production ABDM integration before sandbox/testing/approval.
- Calling a prototype “100% secure” or “fully compliant.”

## 66. What We Should Say to Judges

### Recommended architecture statement:

“MediKiosk is designed as a secure, modular clinical-intake platform. Patient interaction, clinical reasoning support, language services, document intelligence, physician verification and ABDM/HIS interoperability are separated into replaceable modules. Security and privacy controls operate across the entire stack, and the physician remains the final clinical decision-maker.”

### Recommended integration statement:

“Our prototype uses mock/sandbox adapters where live government credentials are not available. The production path is designed around the official ABDM onboarding, testing, security assessment and exit process.”

## 67. Official Sources and Links

### ABDM / NHA

[ABDM official website](#)

[ABDM FAQ — sandbox, consent, integration and security](#)

[ABDM partner/integration information](#)

[ABDM Health Data Management Policy](#)

### Bhashini

[Bhashini API documentation](#)

[Bhashini onboarding and API keys](#)

[Bhashini Pipeline Config](#)

[Bhashini Pipeline Compute](#)

### DPDP / Government of India

[MeitY Acts and Policies](#)

[Digital Personal Data Protection Act 2023](#)

[Digital Personal Data Protection Rules 2025](#)

## 68. Official AI / Development Sources

### Google Gemini

[Google AI Studio](#)

[Gemini API key documentation](#)

[Gemini models](#)

[Gemini 3.8 Flash](#)

[Gemini structured outputs](#)

### Google Stitch

[Stitch](#)

[Google Stitch design announcement](#)

### Google Antigravity

[Antigravity documentation](#)

[Antigravity getting started](#)

### Other LLM providers

[OpenAI business data privacy](#)

[Anthropic tool use](#)

## 69. Final Pre-Development Checklist

- Read the SIH PS again.
- Freeze the architecture.
- Create the Git repository.
- Create the Stitch project.
- Create the Antigravity project.
- Create `.env.example`.
- Create mock providers for ABHA, Bhashini, LLM, OCR and ABDM.
- Create database migrations.
- Create Patient UUID + Visit UUID model.
- Create consent model.
- Create RBAC model.
- Create audit log model.
- Build the first vertical slice.
- Write tests before adding external integrations.
- Never use real patient data in development.
- Document every external credential and where it came from.

## 70. The Exact Prompt We Should Give Antigravity First

Copy and paste this first. Do not ask it to build the whole application yet.

Do not code yet.

Inspect the MediKiosk repository, the SIH26047 Patient Case-Taking Software problem statement, and the engineering architecture guide provided in this workspace.

I want to build a deployable healthcare software system, not a throwaway demo.

For Milestone 1 only, plan this vertical slice:

Patient:

Welcome -> Language -> Mock ABHA -> Consent -> Basic Details  
-> Chief Complaint -> 5-10 deterministic clinical questions  
-> Review -> Submit

Backend:

FastAPI -> PostgreSQL

Doctor:

Login -> Patient Queue -> Open Case -> Structured History  
-> Edit -> Mark Verified

Security:

Patient UUID  
Visit UUID  
RBAC  
object-level authorization  
consent record  
audit logs  
secure session termination  
no secrets in frontend

Do not implement Bhashini, LLM, OCR or live ABDM yet.

First return:

1. architecture plan
2. folder structure
3. database schema
4. API contracts
5. security model
6. frontend pages/components
7. tests
8. implementation order
9. risks
10. acceptance criteria

Do not modify code until I approve the plan.

**After Antigravity gives the plan and you approve it, the next prompt is:**

Implement only the approved Milestone 1 plan.

Do not add extra features.

Do not change the architecture without telling me.

Implement backend, database, frontend and tests together.

After implementation:

- run tests
- show the important diffs
- report what works
- report what is mocked
- report security controls
- report known limitations
- do not deploy

This is where we should begin.